Practical no 1

Problem Statement : Write a program non-recursive and recursive program to calculate Fibonacci numbers and analyze their time and space complexity.

Non-Recursive :-

```
def fibonacci(n):
    a = 0
    b = 1

    print("Fibonacci Series:")

    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b

n = int(input("Enter the number of terms: "))
fibonacci(n)
```

Recursive :-

```
def fibonacci(n):
    if n <= 1:
       return n
    return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter the number of terms: "))
print("Fibonacci Series:")
for i in range(n):
   print(fibonacci(i), end=" ")
```

Theory :

Fibonacci Series is a sequence of numbers in which each number is the sum of the previous two numbers.
The series starts with 0 and 1, and the next numbers are generated by adding the previous two numbers.

Example:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

1. Non-Recursive Method (Iterative Method):

In the non-recursive method, we use loops such as for or while to calculate Fibonacci numbers.
It starts with two variables, 0 and 1, and calculates the next number by adding the previous two numbers.
The values are updated repeatedly until the required number of terms is generated.The non-recursive method
 is used to calculate Fibonacci numbers without calling a function repeatedly. It uses loops such as for or while
 to generate the Fibonacci series. Initially, two variables are assigned values 0 and 1.

The next Fibonacci number is calculated by adding the previous two numbers.
The variable values are updated in each iteration until the required number of terms is generated.
This method is simple, faster, and requires less memory. Its time complexity is O(n) and space complexity is O(1).

2. Recursive Method:

In the recursive method, a function calls itself to calculate Fibonacci numbers.
It uses two base conditions: F(0) = 0 and F(1) = 1.
For other numbers, it calculates the sum of the previous two Fibonacci numbers by calling the same function.
The recursive method is used to calculate Fibonacci numbers by calling the same function repeatedly. In this method,
a function calls itself to calculate the previous two Fibonacci numbers.
It uses two base conditions: F(0) = 0 and F(1) = 1.
If the number is greater than 1, the function calculates the result using the formula F(n) = F(n-1) + F(n-2).
The function continues calling itself until the base conditions are reached.
This method is easy to understand but takes more execution time and memory because it repeats calculations.
Its time complexity is O(2â□¿) and space complexity is O(n).

Method	Time Complexity	Space Complexity
Non-Recursive	O(n)	O(1)
Recursive	O(2â□¿)	O(n)

difference :

Recursive Method	Non-Recursive Method
Uses function calls.	Uses loops.
Requires more memory.	Requires less memory.
Slower for large inputs.	Faster for large inputs.
Uses base conditions.	Uses loop conditions.
Repeats calculations.	Calculates each term once.

Pratical No 2

Problem Statement : Write a program to implement Huffman Encoding using a greedy strategy

```
import heapq
from collections import Counter

class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq
```

```
def huffman_encoding(text):
    frequency = Counter(text)
    heap = []

    for char, freq in frequency.items():
        heapq.heappush(heap, Node(char, freq))

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)

        new_node = Node(None, left.freq + right.freq)
        new_node.left = left
        new_node.right = right

        heapq.heappush(heap, new_node)

    root = heap[0]
    codes = {}

    def generate_codes(node, code=""):
        if node.char is not None:
            codes[node.char] = code
            return

        generate_codes(node.left, code + "0")
        generate_codes(node.right, code + "1")

    generate_codes(root)

    encoded = ""
    for char in text:
        encoded += codes[char]

    return codes, encoded

text = input("Enter a string: ")

if text:
    codes, encoded = huffman_encoding(text)

    print("\nHuffman Codes:")
    for char, code in codes.items():
        print(repr(char), ":", code)

    print("\nEncoded String:")
    print(encoded)
else:
    print("Please enter a non-empty string.")
```

Theory :

Huffman Encoding is a lossless data compression technique used to reduce the size of data by assigning
variable-length binary codes to characters based on their frequency of occurrence. Characters that occur more
frequently are assigned shorter binary codes, while characters that occur less frequently are assigned longer binary codes.

Huffman Encoding uses a Greedy Strategy, in which the two characters with the lowest frequencies are selected and
combined to create a new node. This process is repeated until only one node remains, forming a Huffman Tree. Finally,
binary codes are generated by assigning 0 to the left branch and 1 to the right branch.

Algorithm
1. Calculate the frequency of each character.
2. Create a leaf node for each character.
3. Insert all nodes into a priority queue (min-heap).
4. Select the two nodes with the lowest frequencies.
5. Combine them into a new node.
6. Insert the new node back into the priority queue.
7. Repeat until only one node remains.
8. Generate binary codes using the Huffman Tree.

Complexity	Analysis
Time Complexity	O(n + k log k)
Space Complexity	O(k)

n = Length of input string.
k = Number of unique characters.

Applications
1. File compression.
2. ZIP file compression.
3. Image compression.
4. Data transmission.
5. Multimedia compression.

Practical no 3
Problem Statement : Write a program to solve a fractional Knapsack problem using a greedy method.

```
n = int(input("Enter number of items: "))

weights = []
values = []

for i in range(n):
    w = float(input(f"Enter weight of item {i + 1}: "))
    v = float(input(f"Enter value of item {i + 1}: "))

    weights.append(w)
    values.append(v)

capacity = float(input("Enter knapsack capacity: "))

items = []

for i in range(n):
    if weights[i] <= 0:
        raise ValueError("Weight must be greater than zero")

    ratio = values[i] / weights[i]
```

```
    items.append((ratio, weights[i], values[i]))

items.sort(reverse=True)

total_value = 0

for ratio, weight, value in items:
    if capacity >= weight:
        capacity -= weight
        total_value += value
    else:
        total_value += ratio * capacity
        break

print("Maximum value =", total_value)
```

theory :

The Fractional Knapsack Problem is an optimization problem in which we are given a set of items,
and each item has a specific weight and value (profit). We have to select items to get the maximum
total profit without exceeding the given capacity of the knapsack.In this problem, we can take a complete
item or a fraction (part) of an item. We use the Greedy Method to select items according to their value-to-weight ratio.
Items having a higher value-to-weight ratio are selected first to obtain maximum profit.

How Greedy Method Works
The Greedy Method follows these steps:
1. First, calculate the value-to-weight ratio of each item.
2. Sort all items in descending order according to their ratios.
3. Select the item having the highest ratio first.
4. If the complete item fits in the knapsack, take the whole item.
5. If the item does not fit, take only the required fraction.
6. Continue until the knapsack capacity becomes full.
7. Calculate the total maximum value.

Example
Suppose we have 3 items and the knapsack capacity is 50 kg.

Item	Weight	Value	Value/Weight Ratio
Item 1	10	60	6
Item 2	20	100	5
Item 3	30	120	4

Time and Space Complexity

Complexity	Value
Time Complexity	O(n log n)
Space Complexity	O(n)

Practical no 4
Problem statement : Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy.

```
def knapsack(n, weight, profit, capacity):
```

```
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):

        for w in range(1, capacity + 1):

            if weight[i - 1] <= w:

                dp[i][w] = max(
                    profit[i - 1] + dp[i - 1][w - weight[i - 1]],
                    dp[i - 1][w]
                )

            else:

                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

n = int(input("Enter number of items: "))

weight = []
profit = []

for i in range(n):

    weight.append(int(input("Enter weight of item " + str(i + 1) + ":
")))
    profit.append(int(input("Enter profit of item " + str(i + 1) + ":
")))

capacity = int(input("Enter knapsack capacity: "))

max_profit = knapsack(n, weight, profit, capacity)

print("Maximum Profit =", max_profit)
```

theory :

The Knapsack Problem is an optimization problem in which we are given a set of items,
and each item has a specific weight and profit (value). We have to select items to obtain maximum profit without exceeding the given capacity of the knapsack.
In the 0/1 Knapsack Problem, each item can either be selected completely (1) or rejected (0). We cannot select a fraction of an item.
Dynamic Programming is a method used to solve this problem by dividing it into smaller subproblems and storing their results in a table.
For every item, the algorithm checks two possibilities: either include the item or exclude it. It selects the option that gives maximum profit.
This process continues until all items are checked and the maximum profit is obtained.

Algorithm
1. Start the program.
2. Enter the number of items.
3. Enter the weight and profit of each item.
4. Enter the capacity of the knapsack.

5. Create a DP table and initialize it with zero.
6. Check each item and available capacity.
7. Calculate the profit by including or excluding the item.
8. Store the maximum profit in the DP table.
9. Display the maximum profit.
10. Stop the program.

Complexity	Value
Time Complexity	O(n Ã— W)
Space Complexity	O(n Ã— W)

Where:
- n = Number of items.
- W = Knapsack capacity.

Applications
1. Resource allocation.
2. Budget management.
3. Cargo loading.
4. Investment planning.
5. Project selection.

Practical no 5
Problem statement :Design n-Queens matrix having first Queen placed. Use backtracking to place remaining Queens to generate the final n-queenâ€š matrix.

```
def is_safe(board, row, col, n):

    for i in range(row):
        if board[i][col] == 1:
            return False

    i, j = row - 1, col - 1

    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False

        i -= 1
        j -= 1

    i, j = row - 1, col + 1

    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False

        i -= 1
        j += 1

    return True


def solve(board, row, n):

    if row == n:
```

```
        return True

    if any(board[row]):
        return solve(board, row + 1, n)

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1

            if solve(board, row + 1, n):
                return True

            board[row][col] = 0

    return False

n = int(input("Enter value of N: "))

if n <= 0:
    print("N must be greater than zero.")

else:
    first_row = int(input("Enter first Queen row (1 to N): ")) - 1
    first_col = int(input("Enter first Queen column (1 to N): ")) - 1

    if not (0 <= first_row < n and 0 <= first_col < n):
        print("Invalid Queen position.")

    else:

        board = [[0 for _ in range(n)] for _ in range(n)]

        board[first_row][first_col] = 1

        if solve(board, 0, n):
            print("\nFinal N-Queens Matrix:")

            for row in board:
                print(row)

        else:
            print("\nNo solution exists.")
```

theory :

The N-Queens Problem is a chessboard problem in which N queens must be placed on an N Ã— N chessboard
so that no two queens attack each other. No two queens should be placed in the same row, column, or diagonal.
In this problem, the first queen is already placed at a given position. The Backtracking technique is used to
place the remaining queens one by one in safe positions. Before placing each queen, the algorithm checks whether
 the selected position is safe. If the position is safe, the queen is placed, and the algorithm moves to the next row.
If no safe position is available, the algorithm removes the previously placed queen and tries another position.

This process continues until all queens are successfully placed or no solution exists.

Algorithm
1. Start the program.
2. Enter the value of N.
3. Enter the position of the first queen.
4. Create an N Ã— N matrix initialized with zeros.
5. Place the first queen in the given position.
6. Check whether the next position is safe.
7. If safe, place the queen and move to the next row.
8. If no safe position is available, use backtracking to remove the previously placed queen.
9. Repeat until all queens are placed.
10. Display the final N-Queens matrix.

Complexity	value
Time Complexity	O(N!)
Space Complexity	O(NÂ²)

Applications
1. Artificial Intelligence.
2. Backtracking algorithms.
3. Constraint satisfaction problems.
4. Puzzle solving.
5. Combinatorial optimization.